

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATIONS DEVELOPMENT
IT093IU
LAB 4**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

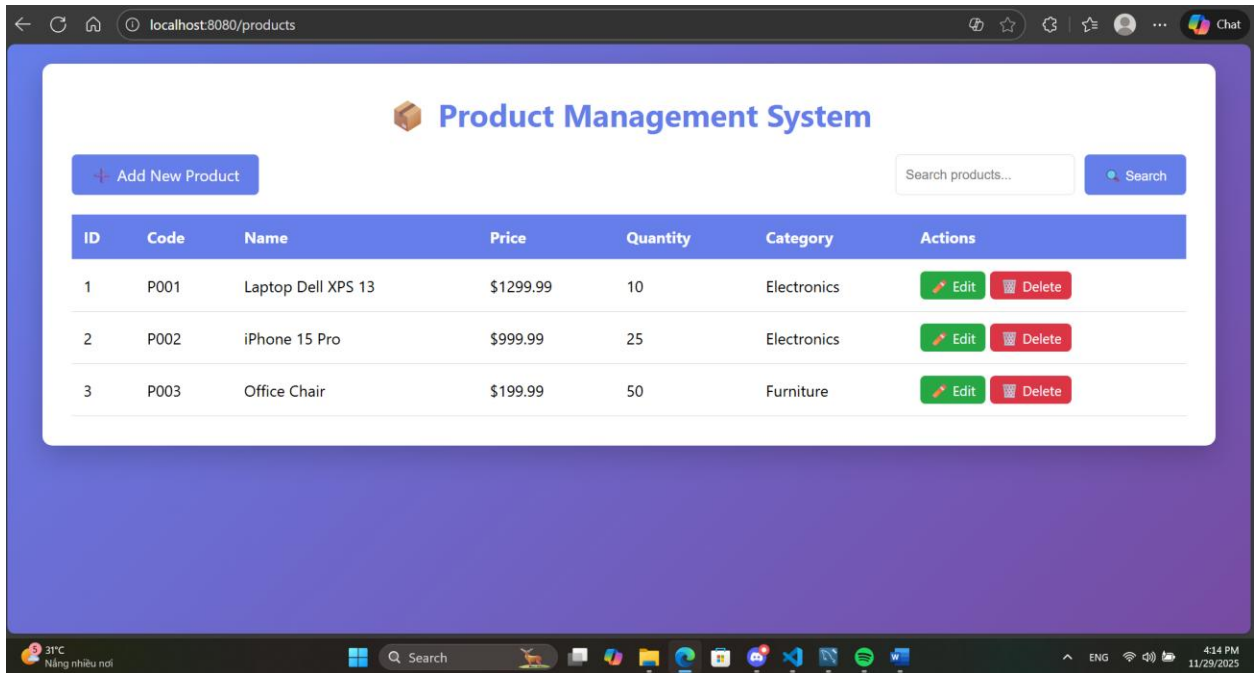
Instructor: MSc. Nguyễn Trung Nghĩa

Contents

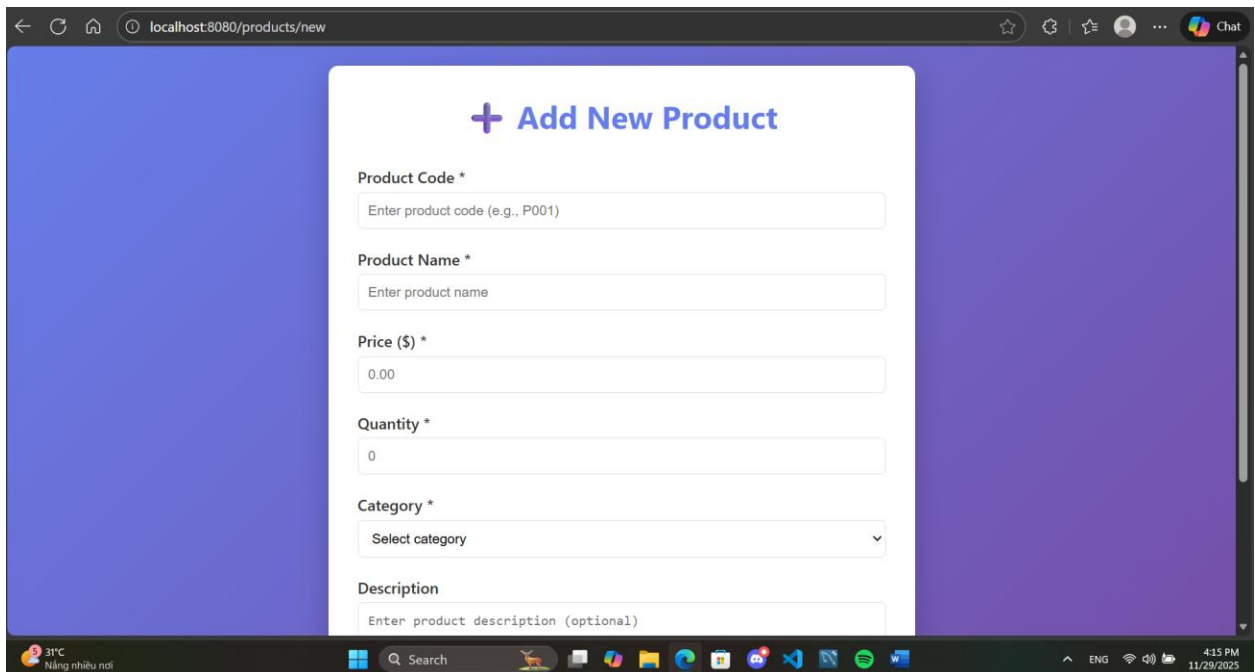
Test cases & demo	3
Code explanation	6

Test cases & demo

- List all products: <http://localhost:8080/products>



- Add new product: <http://localhost:8080/products/new>



localhost:8080/products?sessionId=B228E50D6E23FE8E215C313320647EEE

Product Management System

Product updated successfully!

[Add New Product](#) [Search](#)

ID	Code	Name	Price	Quantity	Category	Actions
1	P001	Laptop Dell XPS 13	\$1299.99	10	Electronics	Edit Delete
2	P002	iPhone 15 Pro	\$999.99	25	Electronics	Edit Delete
3	P003	Office Chair	\$199.99	50	Furniture	Edit Delete
4	P004	Office Desk	\$299.99	36	Furniture	Edit Delete

31°C Năng nhiều nơi

4:17 PM 11/29/2025

- Edit product (ID=1): <http://localhost:8080/products/edit/1>

localhost:8080/products/edit/1

Edit Product

Product Code *

Product Name *

Price (\$) *

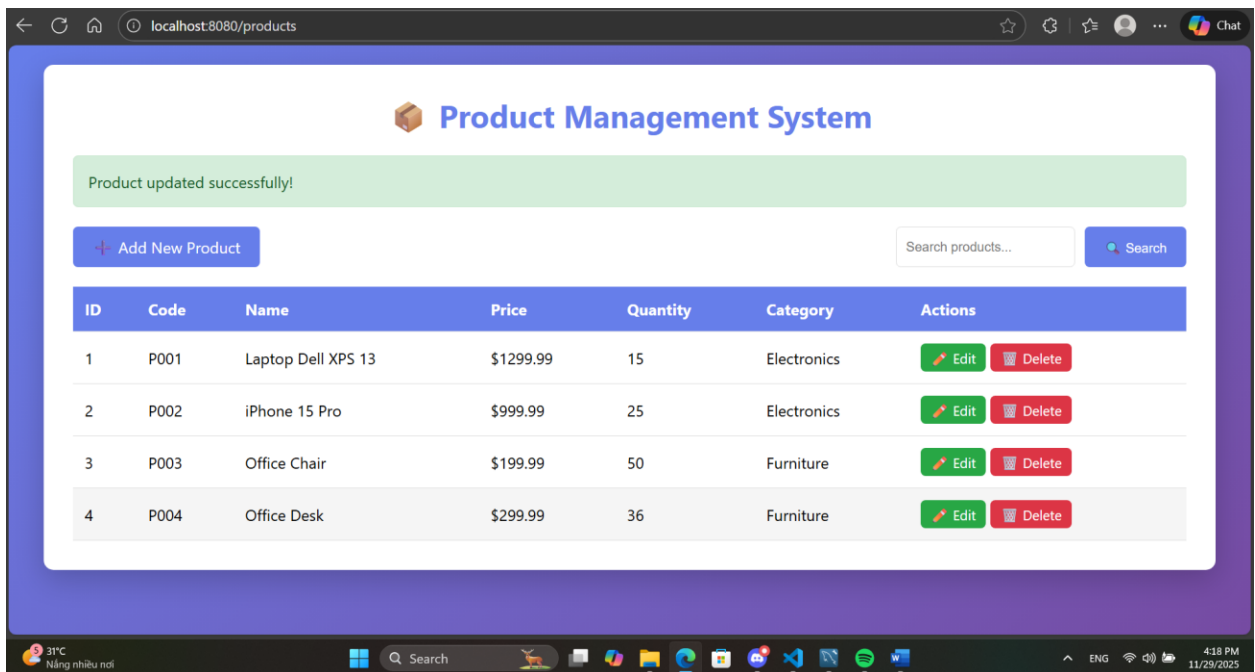
Quantity *

Category *

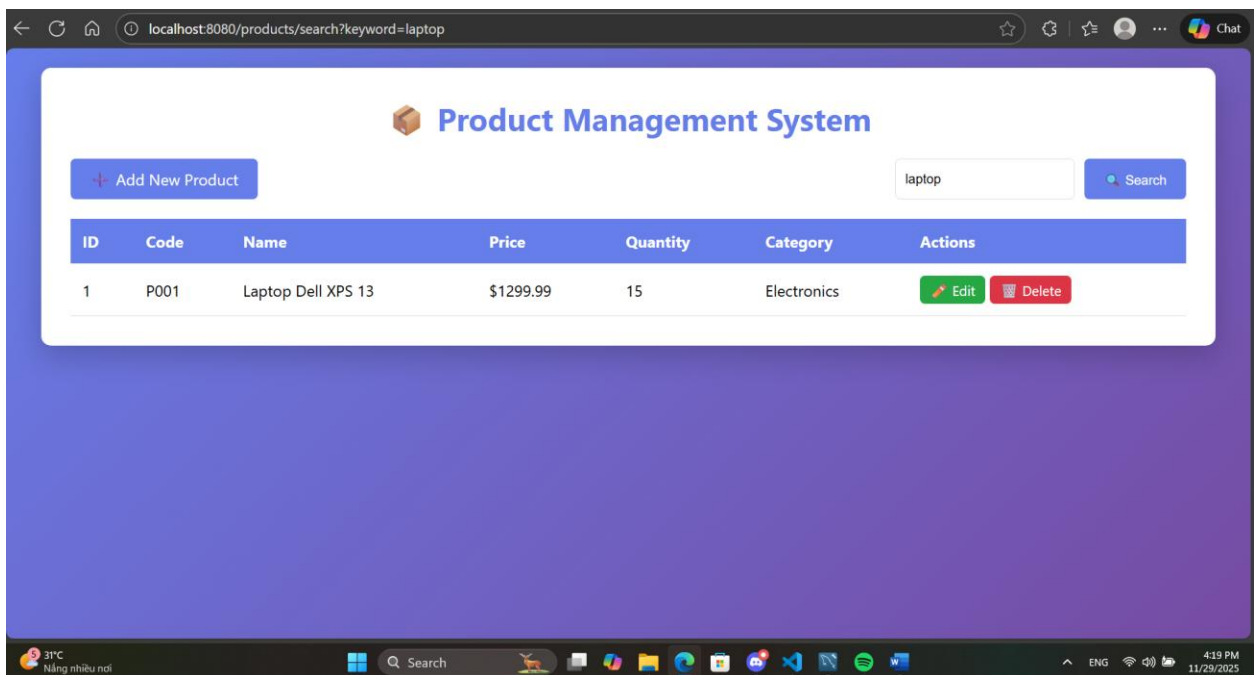
Description

31°C Năng nhiều nơi

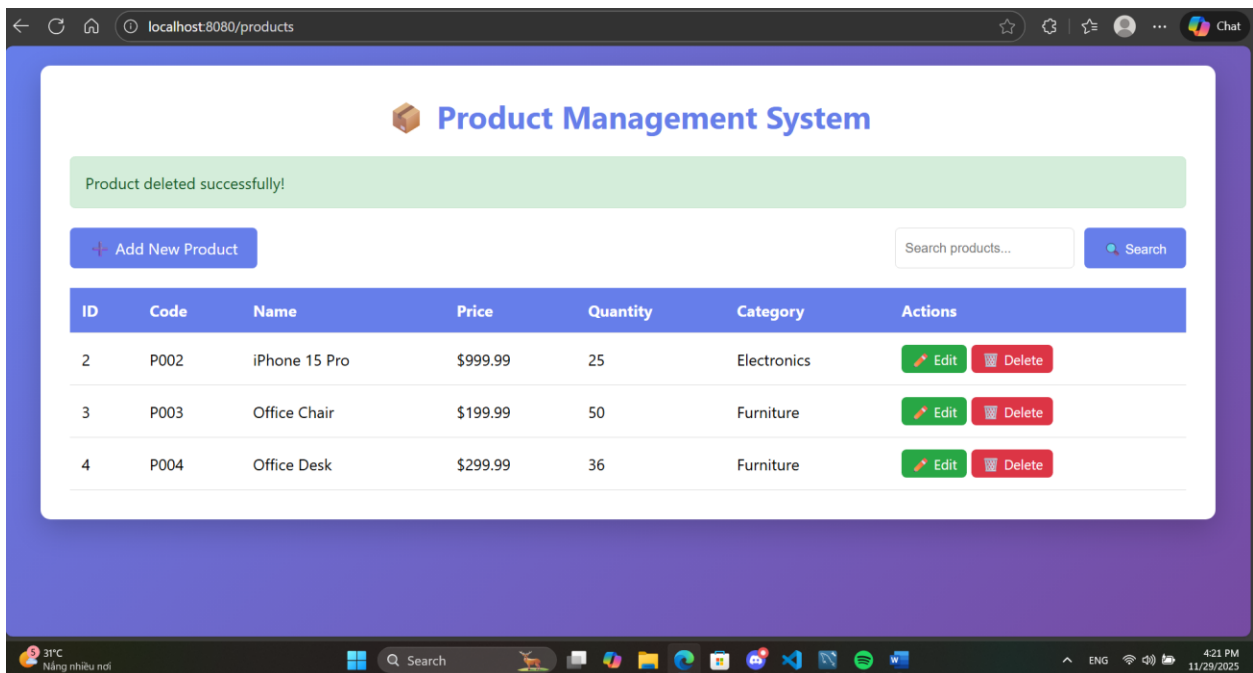
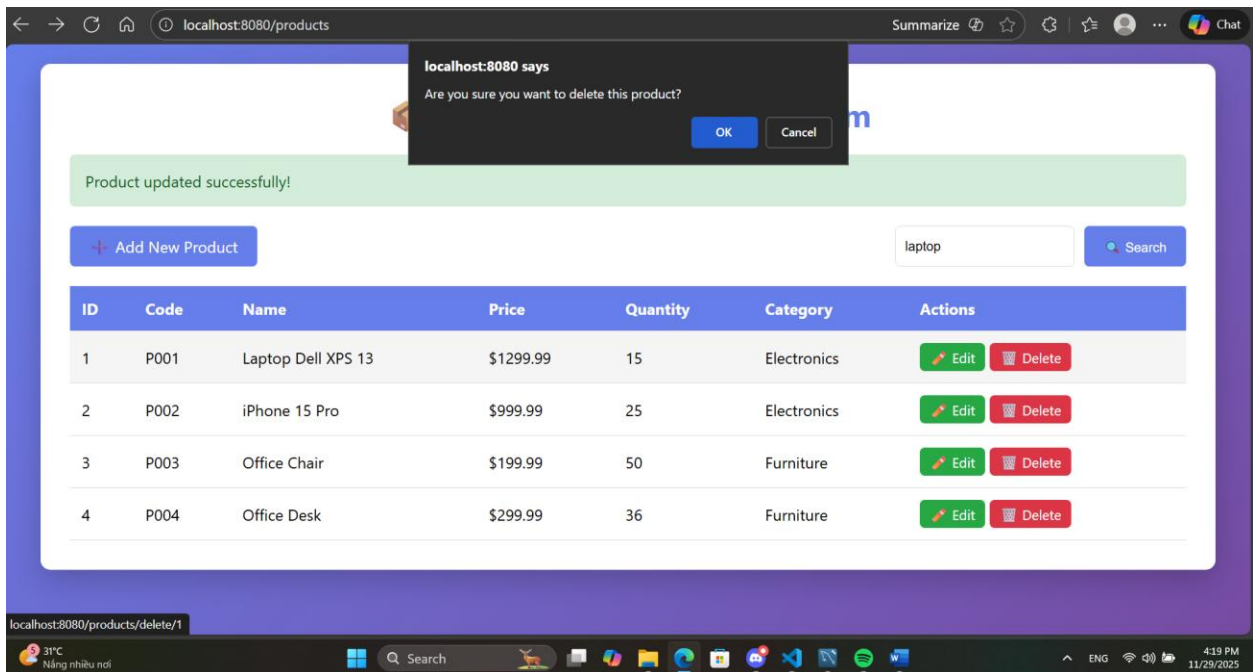
4:18 PM 11/29/2025



- Search products: <http://localhost:8080/products/search?keyword=laptop>



- Delete product (ID=1): <http://localhost:8080/products/delete/1>



Code explanation

1. Product.java (Entity layer)

- Represents a table in the database using `@Entity` and `@Table`.
- Fields map to columns: `id`, `productCode`, `name`, `price`, `quantity`, `category`, `description`, `createdAt`.
- Uses `@Id` and `@GeneratedValue` for primary key.

- Uses constraints like unique, nullable, and length via @Column.
- @PrePersist automatically sets createdAt when inserting a new record.
- Contains getters/setters for JPA field access.

2. **ProductRepository.java (Data access layer)**

- Extends JpaRepository, giving built-in CRUD methods: findAll, save, findById, deleteById.
- Contains custom query methods derived from function names: findByCategory, findByNameContaining, findByPriceBetween, existsByProductCode.
- Spring generates SQL implementations automatically.

3. **ProductService.java (Service interface)**

- Defines business operations: getAllProducts, getProductById, saveProduct, deleteProduct, searchProducts, getProductsByCategory.
- No implementation, only method signatures.
- Provides abstraction between controller and service implementation.

4. **ProductServiceImpl.java (Service implementation)**

- Implements ProductService and contains business logic.
- Uses @Service and @Transactional for service layer control.
- Injects ProductRepository to perform CRUD.
- Implements methods for listing, updating, deleting, searching, and filtering products.
- Adds a logic layer between controller and repository.

5. **ProductController.java (Web controller layer)**

- Handles HTTP requests and returns Thymeleaf HTML views.
- @RequestMapping("/products") maps all product routes.
- Main functions:
 - listProducts(): GET, shows all products.
 - showNewForm(): GET, shows empty form for creation.
 - showEditForm(): GET, loads product for editing.
 - saveProduct(): POST, saves or updates product.
 - deleteProduct(): GET, deletes by ID.
 - searchProducts(): GET, filters by keyword.
 - Passes data to views using Model.
 - Uses RedirectAttributes to show success/error messages.

6. **System flow**

Controller → Service → Repository → Database

- ➔ The controller handles user requests, the service executes business logic, the repository communicates with the database and the Product entity maps data to/from the table.